

TOPIC 01: OOPS IN JS & PROTOTYPAL ARCHITECTURE

Core Purpose of OOP:

Object-Oriented Programming models real-world business domains into modular, reusable objects bundling **State (Properties)** and **Behavior (Methods)** together, eliminating code redundancy (DRY principle).

The 4 Core Pillars of OOP:

1. ENCAPSULATION

2. INHERITANCE

3. POLYMORPHISM

4. ABSTRACTION

Object Prototypes & Prototype Chaining:

In JavaScript, inheritance is **prototype-based**. Every object contains an internal link `[[Prototype]]` (accessible via `__proto__`) pointing to another object from which it borrows methods without duplicating memory.



```
JS prototypes.js
1 const arr = [1, 2, 3];
2 console.log(arr.__proto__ === Array.prototype); // true
3 console.log(Array.prototype.__proto__ === Object.prototype); // true
4 console.log(Object.prototype.__proto__); // null (end of chain)
5
```

TOPIC 02: FACTORY FUNCTIONS VS CONSTRUCTORS & NEW

Factory Functions (Limitation: Memory Waste):

A plain function returning an object. Every instance copies functions into memory separately!

Constructor Functions & The 'new' Operator:

Constructors serve as blueprints. The **new** keyword performs 4 critical steps:

- 1. Creates a brand new empty object `{}`.
- 2. Binds `this` to that newly created object.
- 3. Sets new object's `__proto__` to `Constructor.prototype`.
- 4. Automatically returns `this`.

```
JS constructor.js
1 function Person(name, age) {
2   this.name = name;
3   this.age = age;
4 }
5 // Attach shared method to prototype to save memory
6 Person.prototype.talk = function() {
7   console.log(`Hi, my name is ${this.name}!`);
8 };
9 const p1 = new Person("Shivam", 22);
10 p1.talk(); // Output: Hi, my name is Shivam!
```

TOPIC 03: MODERN ES6 CLASSES SYNTAX

Syntactical Sugar over Prototypes:

Introduced in ES6, `class` provides clean syntax matching OOP languages (Java, C++), but under the hood it remains 100% prototype-based.

```
JS Person.js
1 class Person {
2   constructor(name, age) {
3     this.name = name;
4     this.age = age;
5   }
6   greet() {
7     return `Hello, I am ${this.name}, ${this.age} years old.`;
8   }
9 }
10 const user = new Person("Shivam", 22);
```

TOPIC 04: STATIC METHODS & STATIC PROPERTIES

Class-Level vs Instance-Level Members:

Members declared with `static` belong to the class itself, NOT to instance objects. They cannot be accessed via `this` from an instance.

- **Use Case:** Utility helper functions (e.g. `Math.max()`), factory constructors, database connection singletons.

```
JS MathUtil.js
1 class MathUtil {
2   static PI = 3.14159; // Static property
3
4   static calculateCircleArea(radius) { // Static method
5     return MathUtil.PI * radius * radius;
6   }
7 }
8 console.log(MathUtil.PI); // 3.14159
9 console.log(MathUtil.calculateCircleArea(10)); // 314.159
10 const m = new MathUtil();
11 // m.calculateCircleArea(10); // TypeError: m.calculateCircleArea is not a function
```

TOPIC 05: GETTERS & SETTERS (GET & SET)

Controlled Property Access & Validation:

Getters and Setters allow executing custom logic and validation while accessing or modifying properties as if they were simple fields.

```
1  class Temperature {
2    constructor(celsius) { this.celsius = celsius; }
3
4    get fahrenheit() { // Getter: accessed as a property
5      return (this.celsius * 9/5) + 32;
6    }
7    set celsius(value) { // Setter: validates input
8      if (value < -273.15) throw new Error("Below absolute zero!");
9      this._celsius = value;
10   }
11 }
12 const temp = new Temperature(25);
   console.log(temp.fahrenheit); // 77 (runs getter)
```

TOPIC 06: PILLAR 1 – ENCAPSULATION

Definition & Purpose:

Encapsulation is bundling state and methods inside an object while restricting direct external access to internal implementation details. It prevents accidental mutation and enforces a clean public API.

Modern ES2022 Private Fields (#) & Private Methods:

Prefixing an identifier with # makes it strictly private to the class body. Attempting to access it outside throws a syntax error!

```
JS BankAccount.js
1  class BankAccount {
2      owner;           // Public field
3      #balance = 0;    // Private field (ES2022)
4      #pin;            // Private field
5
6      constructor(owner, initialDeposit, pin) {
7          this.owner = owner;
8          this.#balance = initialDeposit;
9          this.#pin = pin;
10     }
11     // Private Method: Internal validation helper
12     #validatePin(enteredPin) { return this.#pin === enteredPin; }
13
14     withdraw(amount, pin) {
15         if (!this.#validatePin(pin)) throw new Error("Invalid PIN!");
16         if (amount > this.#balance) throw new Error("Insufficient funds!");
17         this.#balance -= amount;
18         return `Withdrew ₹${amount}. New balance: ₹${this.#balance}`;
19     }
20     get balance() { return this.#balance; } // Controlled read-only access
21 }
22 const acc = new BankAccount("Shivam", 5000, 1234);
23 // acc.#balance = 100000; // SyntaxError: Private field '#balance' must be declared in an enclosing class
```

TOPIC 07: PILLAR 2 – INHERITANCE

Definition & Subclassing:

Inheritance allows a child class to inherit attributes and methods from a parent class via extends, promoting code reuse.

- `super()` → Calls the parent class constructor (MUST be called before accessing `this`).
- `super.method()` → Calls parent class implementation directly.

```
JS Inheritance.js
1  class Person {
2      constructor(name, age) { this.name = name; this.age = age; }
3      introduce() { return `Hi, I am ${this.name}.`; }
4  }
5
6  class Student extends Person {
7      constructor(name, age, marks) {
8          super(name, age); // Invoke Person constructor
9          this.marks = marks;
10     }
11     study() { return `${this.name} is studying for exams.`; }
12 }
13 const s1 = new Student("Rahul", 20, 95);
14 console.log(s1.introduce()); // Inherited from Person!
15
16
```

TOPIC 08: PILLAR 3 – POLYMORPHISM

Definition (Many Forms):

Polymorphism allows different child classes to define their own custom implementations of a method that shares the same name in the parent class (Method Overriding).

Dynamic Dispatch & Duck Typing:

In JavaScript, polymorphism is dynamic: if different objects share a method signature, a loop can execute them uniformly without knowing the exact concrete type.

```
JS Polymorphism.js
1  class Animal {
2      speak() { return "Some generic sound"; }
3  }
4
5  class Dog extends Animal {
6      speak() { return "Woof! Woof!"; } // Method Overriding
7  }
8
9  class Cat extends Animal {
10     speak() { return "Meow!"; } // Method Overriding
11 }
12
13 // Polymorphism in action: Uniform execution across heterogeneous array
14 const animals = [new Dog(), new Cat(), new Animal()];
15 animals.forEach(a => {
16     console.log(a.speak()); // Woof! Woof! → Meow! → Some generic sound
17 });
18
19
```

TOPIC 09: PILLAR 4 – ABSTRACTION

Definition: Hiding Implementation Complexity:

Abstraction hides the complex background mechanics and reveals only essential, intuitive interfaces to the user (e.g. you drive a car using pedals without knowing internal combustion physics).

Simulating Abstract Classes in JavaScript:

JavaScript doesn't have an abstract keyword, but we simulate it by checking `new.target` inside the constructor and throwing errors on unimplemented methods.

```
JS AbstractShape.js
1  class Shape {
2      constructor() {
3          if (new.target === Shape) {
4              throw new Error("Cannot instantiate abstract class Shape directly!");
5          }
6      }
7      // Abstract method contract
8      calculateArea() {
9          throw new Error("Abstract method calculateArea() must be implemented!");
10     }
11 }
12 class Circle extends Shape {
13     constructor(radius) { super(); this.radius = radius; }
14     calculateArea() { return Math.PI * this.radius * this.radius; }
15 }
16 // const s = new Shape(); // Error: Cannot instantiate abstract class!
17 const c = new Circle(5);
18 console.log(c.calculateArea()); // 78.539
19
20
```

TOPIC 10: THE 'INSTANCEOF' OPERATOR

Prototype Chain Verification:

The instanceof operator tests whether Constructor.prototype exists anywhere within the prototype chain of an object.

```
JS instanceof.js
1 class Employee {}
2 class Manager extends Employee {}
3 const m = new Manager();
4
5 console.log(m instanceof Manager); // true
6 console.log(m instanceof Employee); // true (inherited!)
7 console.log(m instanceof Object); // true (root prototype)
```

TOPIC 11: COMPOSITION VS INHERITANCE

Architectural Principle:

"Favor object composition over class inheritance."

- Inheritance ("Is-a"): Dog is an Animal. Rigid hierarchy; changes in base class can break all subclasses.
- Composition ("Has-a"): Robot has a Flyer and Cleaner. Flexible; bundles independent functional abilities together.

TOPIC 12: THE 4 PILLARS MASTER SUMMARY TABLE

Pillar	Core Concept	JavaScript Syntax / Mechanism	Primary Benefit
1. Encapsulation	Data hiding & access control	#privateField, #privateMethod(), get/set	Security & prevents unwanted state mutation
2. Inheritance	Code reuse across parent & child	class Child extends Parent, super()	DRY principle & hierarchical modeling
3. Polymorphism	Many forms of same method name	Method overriding & dynamic dispatch	Extensibility & clean unified API loops
4. Abstraction	Hiding internal complexity	Public methods, new.target === AbstractClass	Reduces cognitive load & modular design



Interview Pro-Tip: In JavaScript, classes are syntactical sugar over prototypal inheritance. Remember that private fields (#) are enforced by the JavaScript V8 engine at runtime, unlike TypeScript's private which is only compile-time!